Polymorphism

```
class Animal {

    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {

    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class AnimalMain {

    public static void main(String[] args) {

        Animal animal = new Dog();

        animal.sound();

        Animal cat = new Cat();
        cat.sound();
    }
}
```

final keyword

```
final class Animal {

    // Final Variable
    final String name = "Tom";

    // Final Method
    final void eat() {

        System.out.println("Animal Name : " + name);

        // name = "Max";   // Compile-time Error

        System.out.println("Animal is eating");
    }
}

/*
// Compile-time Error
// Cannot inherit from a final class

class Dog extends Animal {

    @Override
    void eat() {
        System.out.println("Dog is eating");
    }
}
*/

public class AnimalMain {

    public static void main(String[] args) {

        Animal animal = new Animal();

        animal.eat();
    }
}
```

Binding

```java
package com.cait.example1;
//compile time binding
class Animal {
    void speak() {
        System.out.println("Animal speaks");
    }

}

public class AnimalMain {
    public static void main(String[] args) {
        Animal animal = new Animal();
        animal.speak();
    }
}
```

```java
package com.cwt.example6;
//Run time binding
class Animal {

    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {

    @Override
    void eat() {
        System.out.println("Dog is eating");
    }
}
```

```
public class AnimalMain {

    public static void main(String[] args) {

        Animal animal = new Dog();
        animal.eat();
    }
}
```

Upcasting

```
package com.cait.example1;
//upcasting
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
public class AnimalMain {
    public static void main(String[] args) {
        // Upcasting
        Animal animal = new Dog();
        animal.eat();       // Accessible
      //  animal.bark();  // Compile-time error
    }
}
```

Downcasting

```
package com.cait.example2;

class Animal
{
    void eat() {
        System.out.println("Animal is eating");
```

```
    }
}

class Dog extends Animal {

    void bark() {
        System.out.println("Dog is barking");
    }

}

public class AnimalMain {

    public static void main(String[] args) {

        // Upcasting
        Animal animal = new Dog();
        animal.eat();

        System.out.println("----------------");
        //animal.bark();

        // Downcasting
        Dog dog = (Dog) animal;
        dog.eat();
        dog.bark();

    }
}
```

Inheritance flow application

```
class Animal {

    Animal() {
        System.out.println("Animal Default Constructor");
    }
}

class Dog extends Animal {
```

```
    // Default Constructor
    Dog() {
     //is there any constructor calling code?
     //super();
        System.out.println("Dog Default Constructor");
    }

    // Parameterized Constructor
    Dog(String name) {
       //super();
        System.out.println("Dog Parameterized Constructor
: " + name);
    }
}

public class AnimalMain {

    public static void main(String[] args) {

        System.out.println("Creating object using Default
Constructor");
        Dog dog1 = new Dog();

        System.out.println();

        System.out.println("Creating object using
Parameterized Constructor");
        Dog dog2 = new Dog("Tommy");
    }
}
```